Aprendizagem Automática

MEEC

Projeto

Autores:

Francisco Matias (99936) francisco.m.matias@tecnico.ulisboa.pt
Nuno Baptista (96295) nuno.r.baptista@tecnico.ulisboa.pt

2023/2024 – 1º Semestre, P1

Conteúdo

1 Introdução **2**

2 Regression with Synthetic Data **2**
2.1 Q1 - Regressão linear . . . 2
2.1.1 Análise do problema e abordagem utilizada . . . 2
2.1.2 Resultados obtidos e conclusões . . . 2
2.2 Q2 - Regressão linear com dois modelos . . . 3
2.2.1 Análise do problema e abordagem utilizada . . . 3
2.2.2 Resultados obtidos e conclusões . . . 4

3 Image Analysis **4**
3.1 Q3 - Classification - First Task . . . 4
3.1.1 Análise do problema . . . 5
3.1.2 Data Imbalance . . . 5
3.1.3 Abordagem utilizada . . . 5
3.1.4 Resultados obtidos e conclusões . . . 7
3.2 Q3 - Classification - Second Task . . . 8
3.2.1 Análise do problema . . . 8
3.2.2 Data Imbalance . . . 8
3.2.3 Abordagem utilizada . . . 8
3.2.4 Resultados obtidos e conclusões . . . 9

4 Conclusão **10**

1 Introdução

O projeto computacional da disciplina de aprendizagem automática divide-se em duas partes: Regressão e Classificação. O objetivo principal é resolver as tarefas propostas desenvolvendo programas em *Python* que abordem eficazmente os problemas, tendo em consideração os conteúdos programáticos lecionados.

2 Regression with Synthetic Data

2.1 Q1 - Regressão linear

O problema inicial compreende uma questão básica de regressão linear. Propõe-se que se ajuste um modelo linear ao conjunto de training dataset fornecido pelo corpo docente. Este conjunto de dados é composto por $n = 15$ exemplos e é do tipo:

$$\tau_{\text{train}} = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\} \tag{1}$$

onde cada exemplo compreende um vetor de features, $x^{(i)} \in \mathbb{R}^{10}$, com 10 features, e um outcome $y^{(i)} \in \mathbb{R}$, com $i = 1, \ldots, n$.

O objetivo desta tarefa é resolver a questão proposta, minimizando a soma dos erros quadrados (SSE).

2.1.1 Análise do problema e abordagem utilizada

Tendo em conta as características do training dataset fornecido, há aspetos que devemos ter em conta de forma a estimar o modelo que melhor se ajusta aos nossos dados, minimizando o máximo possível o valor de (SSE).

Um dos aspetos que se teve em consideração foi o Cross-Validation ou validação cruzada (CV), que é um procedimento usado para tornar os modelos mais robustos ao overfitting, por meio de treino e teste em vários subconjuntos do conjunto de dados de treino. Além disso, é uma técnica comum para avaliar o desempenho de um modelo e para ajuste de hiperparâmetros, especialmente quando o conjunto de dados é limitado, sendo que neste caso se adequa já que se tem um baixo número de training examples ($n = 15$).

O uso deste método envolve a escolha de um valor para a variável k, que se refere ao número de "folds"ou "partições"nos quais os dados são divididos. Através de testes na implementação realizada, decidiu-se qual o valor de K que produzia melhores resultados, tendo-se chegado à conclusão de que k = 5.

Por fim, testou-se os nossos dados com os três modelos (Linear Regression, Ridge e Lasso), obtendo-se diferentes resultados que se apresentam na tabela 1.

2.1.2 Resultados obtidos e conclusões

De modo a ser possível compararmos os resultados obtidos para cada modelo e, assim, verificarmos qual o que se ajusta melhor aos nossos dados, adicionámos à nossa implementação o cálculo do valor de MSE. Os resultados obtidos foram os seguintes:

Modelo	MSE
Linear Regression	126.83
Ridge	2.96
Lasso	2.99

Tabela 1: Desempenho dos Modelos utilizados

Portanto, da tabela 1 conclui-se que o modelo com o melhor desempenho foi o Ridge, tendo sido o utilizado na nossa implementação final. No entanto, a submissão com esta abordagem apenas obteve a nonagésima sexta (96.º) posição na classificação geral, o que revela que alguns erros foram cometidos na implementação e na abordagem utilizada.

Um dos fatores que pode ter contribuído para esta classificação foi o facto de calcularmos o MSE através de código implementado por nós ao invés de aplicarmos a função mean_squared_error() do python.

Além disso, a implementação poderia ser melhorada adicionando técnicas que aproximassem o modelo tais como a remoção de features irrelevantes ou redundantes.

Por último, a implementação da métrica r^2 para cada modelo poderia ter sido verificada.

Contudo, este resultado fez-nos perceber rapidamente que estávamos a tomar um rumo errado, e com isto vamos começar a procurar mais sobre as melhores ferramentas e métodos para os problemas seguintes sem voltar a cometer os erros desta primeira tarefa.

2.2 Q2 - Regressão linear com dois modelos

A próxima tarefa será também um problema de regressão linear, semelhante ao que foi realizado na secção anterior mas, agora, o training data fornecido pelo corpo docente tem dados que foram produzidos por dois modelos diferentes. O objetivo será tentar separar os dados que foram gerados por cada um dos modelos e tentar ajustar o modelo da melhor forma possível para cada um dos conjuntos.

2.2.1 Análise do problema e abordagem utilizada

Neste momento pretende-se dividir os dois conjuntos de dados que são gerados por dois modelos distintos (modelo com e sem outliers). Esta será a etapa mais trabalhosa da tarefa uma vez que se a estratégia utilizada para dividir os dados não for a mais eficiente, isso pode ser o suficiente para não conseguirmos identificar os modelos corretamente e, por conseguinte, os valores de MSE e de r^2 serão piores que o esperado.

Após ter sido feita uma análise profunda ao problema em questão e variadíssimas propostas de solução do mesmo em grupo terem vindo à tona, decidiu-se que, inicialmente iríamos testar um modelo recursivo, ou seja, primeiramente iríamos analisar cada ponto, um a um, e analisar se a adição desse ponto ao conjunto alterava significativamente a soma do MSE do modelo com inliers e o MSE do modelo com outliers, e caso a soma desse maior que o valor antigo, significava que se tratava de um outlier e como tal guardava-se esse ponto no modelo com outliers, caso contrário guardava-se no modelo "original", o modelo com inliers.

Desta análise resultam dois conjuntos de dados, que foram criteriosamente selecionados de acordo com o parâmetro referido anteriormente, MSE. Desta vez testámos também para o r^2 e os valores estavam de acordo com o pretendido.

Durante o fitting do modelo, adotou-se a mesma abordagem que a utilizada na primeira tarefa, já que tínhamos dois conjuntos de dados e queríamos saber qual o modelo que melhor se adaptava.

Os resultados obtidos com o uso desta abordagem foram os seguintes:

Modelo Inliers	MSE	r^2
Linear Regression	0.075	0.9309
Ridge	0.076	0.9304
Lasso	0.114	0.9025

Tabela 2: Modelo sem Outliers

Modelo Outliers	MSE	r^2
Linear Regression	0.050	0.9496
Ridge	0.058	0.9124
Lasso	0.122	0.8817

Tabela 3: Modelo com Outliers

Após a realização do algoritmo recursivo, começamos a explorar as outras opções que tínhamos pensado, como por exemplo, a realização de um algoritmo que usasse clusters para separar ambos os modelos, como o K-Means, contudo estávamos a ter uns problemas, como por exemplo, na sobreposição de clusters, já que os dois modelos de regressão linear tinham regiões onde os clusters de dados se sobrepunham, o que nos dificultou na tarefa de separação dos pontos de maneira eficaz. Os pontos que pertencem à transição entre as duas regiões de regressão linear podem ser atribuídos a clusters diferentes, levando a resultados imprecisos. Este foi o principal motivo de nos afastarmos por completo deste modelo. Pelos mesmos motivos, acabámos por não usar o Guassian Mixture Model (GMM). Assim, concluímos que os algoritmos de clusterização não eram a solução mais viável para esta tarefa.

Por fim, testámos ainda outro algoritmo, o RANSAC(Random Sample Consensus), um algoritmo que é útil quando se deseja uma regressão robusta a outliers num conjunto de dados. O algortimo seleciona aleatoriamente um pequeno subconjunto de pontos, ajustando um modelo linear a esses pontos, e depois verifica quantos pontos se ajustam bem ao modelo. O processo é depois repetido várias vezes para encontrar o modelo final com o maior número de inliers. Os nossos resultados, após termos testado com todos os modelos possíveis foram ainda piores, por exemplo, usando o RidgeCV, para o modelo com inliers tivemos um r^2 de 0.9018 e para o modelos com os outliers tivemos um r^2 de 0.8971.

Concluindo, optámos por escolher para a implementação final o método recursivo (Recursive residual-based removal), como se pode ver através das tabelas 2 e 3, uma vez que demonstrou um bom desempenho e foi o que produziu melhores resultados.

2.2.2 Resultados obtidos e conclusões

Os resultados acima mostram que os modelos que melhor se ajustam aos nossos dados é o LinearRegression, tendo sido utilizado na nossa implementação final. Efetivamente, na pontuação geral o nosso grupo alcançou a quadragésima quinta (45.º) posição, um resultado melhor que o obtido na primeira tarefa e que indica que fizemos um melhor trabalho. Além disso, desta vez implementámos o cálculo de r^2, e verificou-se qual o valor obtido para cada modelo, o que nos ajudou a ter um melhor desempenho, em relação ao que tivemos na primeira questão.

3 Image Analysis

3.1 Q3 - Classification - First Task

A segunda parte do projeto concentra-se na análise e classificação de imagens médicas de duas dimensões (2D), utilizando um conjunto de dados reais provenientes da base MEDMNIST.

3.1.1 Análise do problema

A primeira tarefa de classificação é binária, e teve como objetivo criar um modelo que preveja se uma imagem do conjunto fornecido é um melanoma (1) ou um nevu (0).

As imagens têm dimensões de 28x28 pixels e possuem 3 canais de cores (RGB), resultando em 2352 elementos em cada entrada (28x28x3). Vale destacar que o conjunto de dados é desequilibrado, pois existe uma grande discrepância entre o número de imagens de melanomas e de nevus no conjunto de treino.

A métrica para avaliarmos o desempenho da nossa implementação será a *Balanced Accuracy*.

3.1.2 Data Imbalance

Como já foi dito, o conjunto de dados está desequilibrado.

Este possui 5358 imagens (85.67%) da classe 0 (nevu) e 896 imagens (14.33%) da classe 1 (melanoma). Com menos exemplos da classe 1 em relação à classe 0, o treino do modelo priorizará a aprendizagem da classe 0. Para resolver esse problema, foram feitas várias tentativas.

É importante notar que todas as experiências tentadas, descritas nas próximas seções, foram aplicadas apenas após a divisão do conjunto de treino, validação (85% e 15%, respectivamente) e apenas no conjunto de treino.

3.1.3 Abordagem utilizada

Começou-se por fazer uma pesquisa de quais os melhores classificadores para as tarefas de classificação, os quais vamos referir mais à frente. Além disso, como referido anteriormente, temos que ter em atenção o *imbalance* do conjunto de dados. De seguida apresentam-se algumas técnicas utilizadas e testadas para resolvermos estes problemas.

Subsampling e Oversampling

A primeira abordagem considerada para lidar com a diferença no número de dados nos conjuntos foi a subamostragem da classe maioritária, que envolve a remoção de algumas imagens. No entanto, devido à limitada quantidade de dados, essa estratégia resultou num desempenho insatisfatório. Isso ocorreu porque a subamostragem levou à exclusão de imagens essenciais, tornando o nosso modelo inadequado para os dados fornecidos.

Em contrapartida, explorámos a técnica de sobreamostragem, que envolveu o uso de duas abordagens: SMOTE e ImageDataGenerator. O método SMOTE é um algoritmo que gera dados sintéticos ao sintetizar pontos de dados com base nos k-vizinhos mais próximos, visando equilibrar a distribuição das classes e melhorar o desempenho do modelo. Além disso, implementámos o ImageDataGenerator para adicionar cópias modificadas das nossas imagens originais (aquelas da classe minoritária) ao conjunto de dados. As modificações incluíram rotação, deslocamento de largura/altura, mudança de brilho e inversão horizontal/vertical.

Estas técnicas de aumento de dados ajudaram a enriquecer o nosso conjunto de dados, tornando-o mais diversificado e robusto, o que por sua vez melhora a capacidade do nosso modelo de lidar com a classe de dados que está presente em minoria.

Class weights

Além das técnicas de aumento de dados mencionadas anteriormente, também exploramos um método adicional que não envolveu a geração de novos dados. Essa abordagem envolveu a atribuição de pesos

de classe aos modelos de classificação. Ao implementar esta estratégia, atribuímos um peso mais elevado à classe em minoria, resultando numa função de perda que se torna a média ponderada das perdas previamente calculadas. Essa abordagem visa destacar a importância da classe minoritária, incentivando o classificador a minimizar ainda mais os erros associados a essa classe, apesar do seu número limitado de amostras.

Arquitetura do Modelo

No que diz respeito à escolha do modelo teve-se várias opções em cima da mesa. A primeira opção foi o modelo SVM (Support Vector Machine), que rapidamente concluimos que não se adequava ao nosso problema já que este tipo de redes neuronais não são capazes de aprender automaticamente a partir das imagens, tornando o processo demorado e suscetível a erros na extração de recursos relevantes em imagens médicas.

Outra hipótese seria o modelo KNN (K-Nearest Neighbors), mas também não se revelou uma boa opção já que o KNN requer a representação das imagens como vetores de características e calcula a proximidade com base em distâncias euclidianas, o que pode não ser adequado para dados de imagens de alta dimensão.

Em relação ao modelo LR (regressão logistica), vimos que também não teria um bom desempenho já que é mais apropriado para tarefas de classificação binária em dados tabulares, e não para imagens deste tipo.

Portanto, chegou-se à conclusão que o modelo mais adequado ao nosso problema seria o CNN (Redes Neurais Convolucionais), já que este tipo de redes neuronais conseguem aprender automaticamente as características das imagens, resultando num melhor desempenho e maior eficiência. De seguida iremos dizer as alterações que fizemos à nossa implementação com o CNN.

Começamos com o pré-processamento dos dados, aplicando inversões aleatórias para aumentar a variabilidade e rotações para garantir que a rede aprenda independentemente da orientação.

Na etapa de extração de características, utilizamos três convolutional layers com 28 kernels de 3×3 e funções ReLU para modelar características nas imagens. Esta etapa ajuda a identificar padrões visuais nas imagens que são importantes para a classificação. Reduzimos ainda a dimensionalidade com três camadas de maxpooling, intercaladas entre as convolutional layers, mantendo as informações mais importantes das imagens.

Em seguida, preparamos os dados com uma camada de achatamento e aplicamos três camadas totalmente conectadas com funções ReLU. O número de neurónios varia em cada camada (1024, 256 e 56) e usamos dropout para evitar overfitting.

A camada de saída possui 2 neurónios e utiliza a função softmax para classificação binária. Isto significa que o modelo é treinado para prever se uma dada imagem corresponde a um melanoma ou a um nevu.

Para otimizar o treino, implementámos o optimizador Adam com um *Learning Rate* de 0,0002, visando minimizar a função de perda de entropia cruzada binária. Devido à complexidade da arquitetura da CNN, o overfitting é um problema que temos de ter em conta. Isto ocorre quando a perda de treino diminui, mas a perda de validação começa a aumentar após um ponto crítico. Para combater o overfitting, implementamos a técnica de paragem antecipada (Early stopping), com um limiar de 20 epochs para a perda de validação. Em outras palavras, quando a paragem antecipada é ativada, os pesos finais da rede correspondem à menor perda de validação.

Algortimo testado

O algoritmo implementado é bastante simples. Tem a estrutura de uma rede neuronal especificada em cima que irá ser usada para verificar se uma dada imagem é referente a Nevu ou Melanoma. De seguida descarregam-se os dados fornecidos para vetores e, através do train_test_split() dividimos os dados em dados para treino e dados para a validação do modelo. Em relação aos dados de treino enviámos os dados para o smote, enquanto que os dados de validação fizemos uma redução de imagens através do algoritmo Subsampling(), de forma a deixarmos de ter a imbalance nas duas classes. No fim treinámos o modelo e demos *predict* para obtermos o ficheiro de saída.

3.1.4 Resultados obtidos e conclusões

Considerando as informações previamente mencionadas, procedeu-se à implementação das diversas soluções propostas e verificou-se os respetivos resultados produzidos:

CNN	Balanced Accuracy
Subsampling	0.8178
SMOTE	0.8571
Data Augmentation	0.8413

Tabela 4: Desempenho dos Modelos utilizados

Chegamos à conclusão de que a combinação mais eficaz foi a Rede Neural Convolutiva (CNN) em conjunto com o modelo SMOTE. Esta foi a configuração escolhida para a implementação final e, de facto, os resultados obtidos foram bastantes positivos como se pode observar nos gráficos seguintes:
1

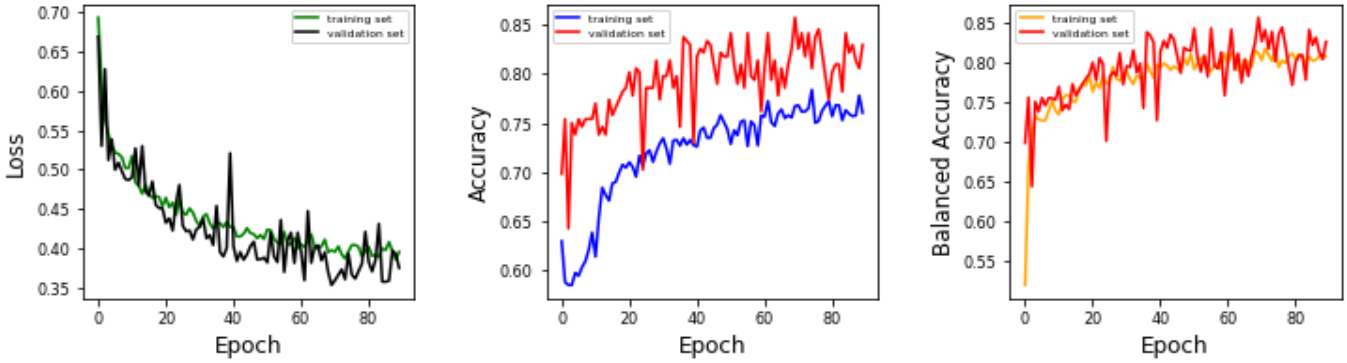

Figura 1: Training e validation loss/accuracy/balanced accuracy, em função do número de epochs

Observando os gráficos obtidos acima, podemos concluir que os nossos resultados estão dentro do esperado, já que o gráfico de *Loss* em função do número de *epochs* mostra uma diminuição constante da perda à medida que as *epochs* avançam, convergindo para um valor minimo, o que indica que o modelo está a aprender eficazmente e a melhorar o seu ajuste de dados. Além disso, o gráfico de *Balanced Accuracy* em função do número de *epochs* mostra um aumento e consequente estabilização da *accuracy* à medida que as *epochs* avançam, o que também indica que o modelo é capaz de fazer previsões precisas para todas as classes do problema. É importante referir que a proximidade entre as curvas do validation set e do training set é um sinal positivo de que a implementação está a ter um bom desempenho.

Efetivamente, com esta implementação alcançou-se a décima primeira (11.º) posição na classificação geral, um resultado que nos indica que as soluções que encontrámos para esta tarefa foram eficientes e obtiveram excelentes resultados.

3.2 Q3 - Classification - Second Task

O objetivo da segunda parte da tarefa de classificação é semelhante ao da primeira parte. Neste caso, pretende-se classificar imagens 2D em seis classes diferentes, utilizando um conjunto de dados reais provenientes da base MEDMNIST, constituídos por dois conjuntos de dados distintos: dermatoscópicas e de microscopia de células sanguíneas.

3.2.1 Análise do problema

A segunda tarefa de classificação, teve como objetivo criar um modelo que previsse se uma imagem do conjunto fornecido é um nevu(0), melanoma(1), lesões vasculares(2), granulocytes(3), basophils(4) ou lymphocytes(5).

As imagens têm dimensões de 28x28 pixels e possuem 3 canais de cores (RGB), resultando em 2352 elementos em cada entrada (28x28x3). É de destacar que o conjunto de dados é desequilibrado, já que existe uma grande discrepância entre o número de imagens das 6 classes no conjunto de treino.

A métrica para avaliarmos o desempenho da nossa implementação será a *Balanced Accuracy*.

3.2.2 Data Imbalance

Como já foi dito, o conjunto de dados está desequilibrado.

A Dermoscopy dataset possui 6368, das quais 5362 imagens (84.20%) da classe 0 (nevu), 890 imagens (13.97%) da classe 1 (melanoma) e 116 imagens (1.822%) da classe 2(lesões vasculares).

A BloodCell dataset possui 4268 imagens, das quais 2305 imagens (54.04%) da classe 3 (granulocytes), 990 imagens (23.19%) da classe 4 (basophils) e 966 imagens (22.63%) da classe 5 (lymphocytes).

Para combater estas diferenças foram implementados vários métodos, como por exemplo, o uso de Class Weights, para as classes com menos amostras terem uma preponderância maior ao treinarmos o modelo.

É importante notar que todas as experiências tentadas, descritas nas próximas seções, foram aplicadas apenas após a divisão do conjunto de treino, validação (85% e 15%, respectivamente) e apenas no conjunto de treino.

3.2.3 Abordagem utilizada

Uma vez que este problema é idêntico ao primeiro, apenas testou-se um classificador para as tarefas de classificação (CNN) pelos mesmos motivos descritos acima. Além disso, como referido anteriormente, temos que ter em atenção ao *imbalance* do conjunto de dados.

As técnicas utilizadas e testadas para resolvermos o problema são as mesmas já apresentadas, uma vez que os problemas são idênticos. Assim, e dada a falta de espaço, não voltaremos a referir o mesmo.

Arquitetura do Modelo

A arquiteura deste modelo é praticamente idêntica à da primeira tarefa de Classificação, com a diferença que a nossa rede neuronal nesta tarefa tem 6 neurónios (6 output layers ,um para cada classe), ao invés de 2 neurónios (2 output laywers).

Algoritmos testados

1º modelo

Sucintamente, aplicámos uma única rede neuronal com a estrutura especificada na secção "Arquitetura do modelo". Seguidamente carregámos os dados fornecidos pelos professores e separámo-los em dois vetores, cada um com dados de cada dataset, para serem depois testados através da função train_test_split().

Após termos os dados de treino, aplicámos o smote aos dados de treino de cada dataset e no fim concatenámos os vetores (X e Y) de forma a podermos treinar o modelo corretamente. No que toca aos dados de validação, foi-nos recomendado a não alterá-los e portanto seguimos a recomendação.

No fim, já com o nosso modelo treinado, para obtermos o ficheiro de saída aplicá-mos aos dados de teste o *predict*.

De realçar que o mesmo algoritmo foi posto à prova com todas as estratégias possíveis e não apenas o SMOTE, como podemos verificar pela tabela 5

2º modelo

Para além da abordagem especificada em cima, o nosso grupo implementou outra que consistia na separação de dados através da rede neuronal da terceira task (esta que tinha dois output layers) com recurso a uma estratégia de *Binary Classification* que separava os dados recebidos (*X_train*) em duas datasets (A dataset de Dermoscopy ficava com *0´s* enquanto que a dataset de BloodCell ficava com *1´s*).

Após a divisão, os dados ficaram separados por duas redes neuronais (uma para cada dataset cada uma com 3 output layers, ou seja, a Dataset de Dermoscopy terá 0,1 e 2 como output e a Dataset BloodCell terá 3,4,5 como output) e, de seguida, treinou-se os dados de cada rede para se prever a resposta final.

Desta forma, o nosso programa, consoante a respetiva imagem de entrada, primeiramente decidia a que dataset a imagem pertencia e só depois decidia a que classe a imagem melhor se enquadrava.

Optámos por não submeter este algoritmo já que após aplicarmos todos os testes possíveis, o máximo de *Balanced Accuracy* que obtivemos foi, aproximandamente, 0.8237.

De realçar que o mesmo algoritmo foi posto à prova com todas as estratégias possíveis e não apenas o SMOTE, como podemos verificar pela tabela 6

3.2.4 Resultados obtidos e conclusões

De forma a concluirmos qual a solução mais adequada ao nosso problema, efetuámos diversos testes e, através da métrica *Balanced Accuracy*, comparámos as diversas opções e chegámos às seguintes conclusões:

CNN (1º algortimo)	Balanced Accuracy
Subsampling	0.7935
SMOTE	0.8794
Data Augmentation	0.8567

Tabela 5: 1 rede neuronal apenas (6 output layers)

CNN (2º algoritmo)	Balanced Accuracy
Subsampling	0.7732
SMOTE	0.8237
Data Augmentation	0.8172

Tabela 6: 1 rede neuronal (2 output layers) + 2 redes neuronais (3 output layers/cada)

Concluímos assim que a combinação CNN + SMOTE é novamente a que produziu melhores resultados de *Balanced Accuracy*, sendo a que acabou por ser implementada. Porém, nesta fase, assumimos o mesmo tipo de rede neuronal (CNN), já que o problema era idêntico ao da terceira tarefa e já tínhamos estudado anteriormente que a rede neuronal que se adequava melhor ao nosso problema era o CNN. É de referir que nesta fase não nos era recomendado o uso do SMOTE contudo, como tínhamos layers para Data Augmentation como o random flip e o random rotation, acabou por ser essa a melhor implementação possível de entre as testadas.

De seguida, com o código implementado, observou-se o comportamento do nosso modelo através dos seguintes gráficos obtidos com o mesmo:

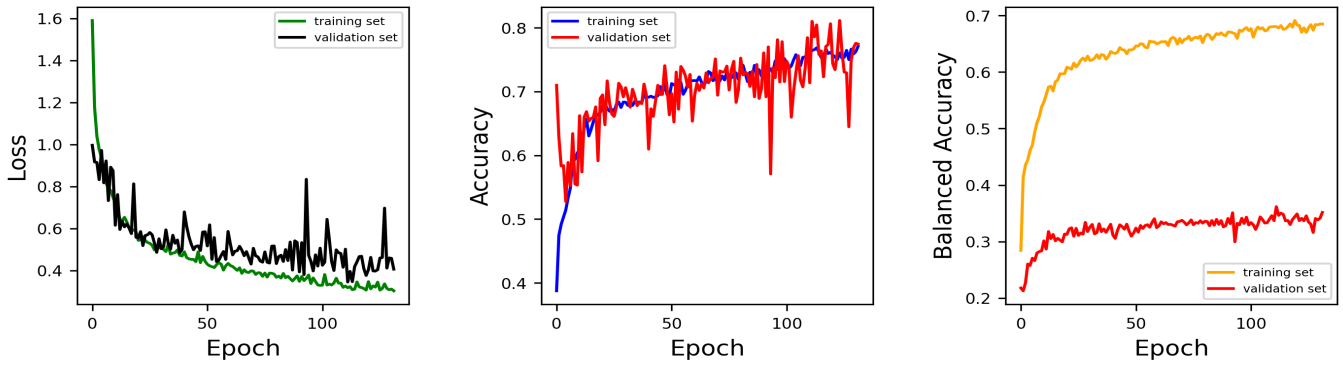

Figura 2: Training e validation loss/accuracy/Balanced Accuracy, em função do número de epochs

Através da análise dos gráficos acima verificamos novamente que o gráfico de *Loss* em função do número de *epochs* está dentro do esperado uma vez que a curva mostra uma diminuição constante da perda à medida que as *epochs* avançam, convergindo para um valor mínimo.

De igual modo, o gráfico de *accuracy* em função do número de *epochs* também está como esperado, já que a *accuracy* aumenta à medida que o número de *epochs* aumenta.

O último gráfico foi o que suscitou maior dúvidas e o que foi alvo de uma maior discussão por parte do nosso grupo. O motivo foi que as curvas não estão como esperávamos, e uma das razões poderá ser que este não corresponde à realidade do nosso algoritmo, derivado a um erro não identificado na função BalancedAccuracy implementada por nós apenas para ser plotted, algo que na tarefa anterior não nos aconteceu. Contudo, o desempenho do mesmo não influenciou em nada a nossa decisão acerca da boa implementação do modelo, já que os outros dois gráficos estão de acordo com as expectativas e o erro é localizado numa só função.

Contudo, na classificação geral, o resultado obtido foi a quadragésima nona (49.º) posição, um resultado bastante positivo mas que nos indica que a nossa solução poderia ser ainda melhor, de modo a obtermos melhores resultados. Talvez utilizar outros modelos ou métodos na nossa implementação seria uma forma de obter uma melhor pontuação.

4 Conclusão

O projeto em que estivemos envolvidos abrangeu uma variedade de tarefas desafiadoras no campo da aprendizagem automática, incluindo regressão linear e classificação de imagens médicas. Ao longo desse processo, pudemos aprimorar as nossas competências e expandir os nossos conhecimentos.

Inicialmente, na primeira tarefa, enfrentámos desafios com os algoritmos de regressão linear, obtendo

um desempenho abaixo do esperado. No entanto, essa experiência foi fundamental para reconhecermos que precisávamos de aprimorar a nossa abordagem de trabalho. Isso motivou-nos a realizar pesquisas mais aprofundadas sobre métodos e algoritmos eficazes para as tarefas que tínhamos pela frente.

Ao longo do projeto, ajustámos a nossa estratégia e passámos a adotar métodos alternativos que levaram a resultados mais satisfatórios. Este resultado ensinou-nos a valorizar a eficácia dos métodos, algoritmos e modelos que aprendemos nas aulas teóricas, quando aplicados em cenários do mundo real.

Vale destacar que o projeto consistiu em quatro partes distintas. Inicialmente, enfrentámos duas tarefas de regressão linear, sendo a primeira uma regressão linear simples, contudo, não é possível tirarmos muitas conclusões sobres os algoritmos abordados devido aos problemas expostos acima.

Relativamente à segunda tarefa de regressão linear, concluímos que o melhor algoritmo a ser utilizado era o método iterativo, uma vez que era o que nos dava mais flexibilidade na modelagem do processo de remoção de outliers e na adaptação do modelo. Principalmente, escolhemos este método já que tem uma adaptação continua, ou seja, o método permite que o modelo se ajuste continuamente à medida que os outliers são removidos, o que pode ser benéfico quando não se tem conhecimento prévio sobre a distribuição e a extensão dos outliers no conjunto de dado, como no nosso caso.

Em seguida, na segunda parte do projeto, terceira e quarta tarefas, enfrentámos problemas de classificação de imagens médicas.

A terceira tarefa envolveu uma classificação binária, com duas classes, enquanto que a quarta foi uma tarefa de classificação com seis classes distintas.

Durante a segunda metade do projeto, utilizamos redes neurais convolucionais (CNN) para realizar a classificação de imagens médicas e também empregámos a técnica SMOTE para lidar com desequilíbrios nos dados. Após terminados estes problemas de classificação, concluímos que o SMOTE é a escolha mais apropriada quando se lida com conjuntos de dados altamente desequilibrados, como é o nosso caso. Ele cria amostras sintéticas da classe minoritária, o que ajuda a equilibrar as classes e a melhorar a capacidade do modelo de generalização. Esta escolha faz com que abordemos o problema de desequilíbrio de classes de forma direta e eficaz. Igualmente, concluimos que as redes neuronais CNNs são altamente eficazes na extração de recursos de imagens, tornando-as na escolha ideal para tarefas de visão computacional, como classificação de imagens. Tal facto levou-nos a escolher este tipo de redes neuronais já que estas são projetadas especificamente para extrair características visuais e têm sido muito bem-sucedidas em uma ampla gama de problemas de classificação de imagens, como é o nosso desejo.

No geral, este projeto representou uma oportunidade valiosa para aplicar os nossos conhecimentos teóricos num contexto prático e desafiador, aprimorando assim a nossa compreensão e competências em machine learning.